Glucometer Sensor - Final Project

Naiara Aznar Calvo*, Patricia González Gil†
CMPE3815A: Microcontroller Systems
University of Vermont
Email: *naznarca@uvm.edu †pgonzal3@uvm.edu

***Abstract*—The idea of our project proposal is to program, design, and build a glucometer sensor which can be used to obtain a person's glucose levels in real time. We aim to make a fully functional sensor which has to read a blood sample from a reactive probe and processes the sample to obtain an indirect measurement of the person's blood glucose levels. Then, this result has to be shown on an OLED display screen. We will need components such as 0.1µF capacitors, 2200pF ceramic capacitors, an analog-to-digital converter, an Op-Amp, and test stand PCB clips.**

MATERIALS

- Jumper wires
- Arduino Nano board
- Breadboard
- 10 K Ohm resistors (3)
- 47 M Ohm resistors (3)
- 0.1µF capacitors (2)
- 2200pF ceramic capacitor (1)
- Test stand PCB clips 2.54 mm (2)
- OLED Display
- Operational Amplifier (Op-Amp), we used the Op Amp IC LM741 model
- Analog-to-Digital Converter (ADC), we used the internal one from our Arduino Nano board
- reactive probe or strip from a diabetes kit

All these components are available at the lab inventory. We had to purchase the test stand for PCB clips.

TASK 1. BACKGROUND INFORMATION

Glucometer sensors are devices used by diabetic patients to help them read their blood glucose levels out of a blood sample in a reactive probe. The sensor inside a glucometer is in charge of signal conditioning and analog-to-digital conversion of the current that is obtained from the reactive probe. The resulting voltage is calibrated so that it matches a certain set value of blood glucose.

The reactive probe or glucose test strips have two main electrodes: the working electrode (WE) and the reference electrode (RE). The chemical reaction that generates a measurable current occurs at the WE, while the RE is supplied with a steady voltage for a fixed electrochemical potential.

The current generated by the WE has to be converted to a readable voltage, so we need an Analog-to-Digital Converter (ADC) for that. The Arduino Nano board we used already had an internal ADC.

The chemical reaction gives a current that is too low to be read, so we also need an operational amplifier, configured as a Transimpedance amplifier (TIA). We used the IC LM741 op amp. A TIA converts the WE current into a proportional voltage.

Lastly, we used a few capacitors to reduce the noise and high-frequency oscillations, and stabilize the signal.

TASK 2. ARDUINO CONNECTIONS

Table 1 shows the connection pins of the test strip to the clip and to the Arduino board.

TABLE I: Connections between test strip - clip - Arduino Board

Test strip pins	Clip pins	Arduino pins	OLED pins	Opamp
x	x	GND	GND	
x	x	Vin	Vcc	+V
Re	4	x	x	+IN
We	2	x	x	x
GND	3	x	x	x
x	x	A5	SCL	x
x	x	A5	SDA	x

The rest of the connections can be found in figure 1, which shows the schematic of the whole circuit.

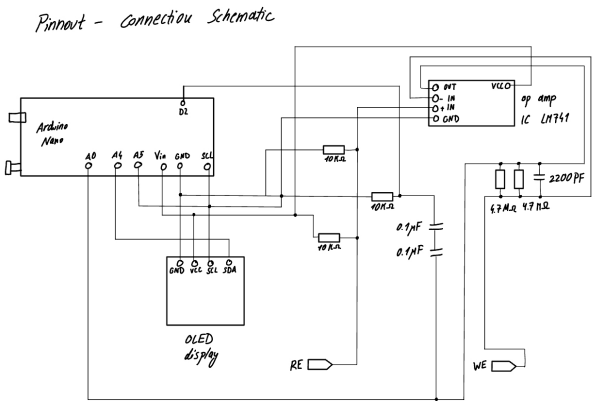

Fig. 1: Circuit Schematic

Task 3. Setup and Sensor Communication

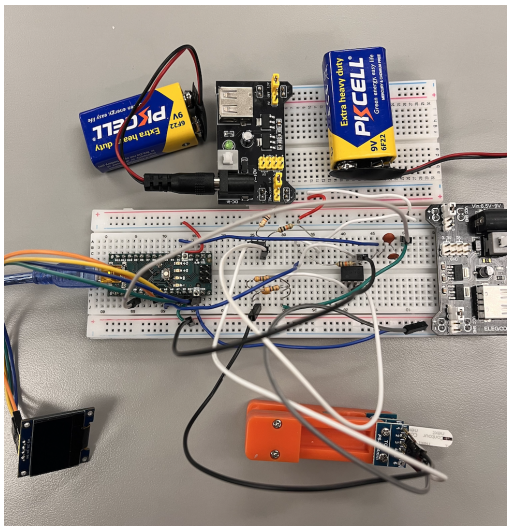

Fig. 2: Glucometer Setup

Figure 2 shows the circuit setup with all the circuit components connected to the reactive strip, which can be seen more clearly in Figure 3, the OLED, shown in Figure 4, the Arduino Nano board, and the power supplies.

Reactive Probe Setup

The width of the probe is 7 mm, and each of the 3 strips was measured to be 2 mm wide. We are going to use a 5 pin clip, which has a spacing of 2.54 mm, and because it is not completely evenly spaced for our test strip, we will bend the external pins to avoid contact problems if necessary.

Task 4. Programming

The code that designs and calibrates the glucometer is in listings 1-3.

```
//              GLUCOMETER SENSOR              //
//Microcontroller Systems Final Project//

//This is the code we are using to measure the current from the reactive probe and calibrating it to obtain a measurable blood glucose value.
//We are compiling and trying this code on the 5th of May.
#include <Wire.h>
#include <U8g2lib.h>

#define SENSOR A0

U8G2_SSD1306_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, U8X8_PIN_NONE);

// Calibration:
// We will use the following expression to obtain glucose levels out of voltage values:
//glucose = voltage_mV * SLOPE + OFFSET, as the measured voltage and blood glucose values follow an almost-linear relationship.

//These are two reference points
float V1 = 350.0; //mV low
float G1 = 70.0; //mg/dL low

float V2 = 900.0; //mV high
float G2 = 180.0; //mg/dL high

float SLOPE;
float OFFSET;

float readVoltage_mV(byte samples) {
  long sum = 0;

  for (byte i = 0; i < samples; i++) {
    sum += analogRead(SENSOR);
    delay(5);
  }

  float raw = sum / (float)samples;
  return raw * (1100.0 / 1023.0); //we are going to work with 1.1V because Arduino Nano uses this internal reference voltage.
}

float voltageToGlucose(float voltage_mV) {
  return voltage_mV * SLOPE + OFFSET;
}

const char* classify(float g) {
  if (g < 70) return "LOW";
  if (g < 100) return "NORMAL";
  if (g < 200) return "ELEVATED";
  return "HIGH";
}

//This code is for the interface and visualization in the OLED display screen.
void drawText(const char* l1, const char* l2) {
  u8g2.firstPage();
  do {
    u8g2.setFont(u8g2_font_6x10_tr);
    u8g2.drawStr(0, 16, l1);
    u8g2.drawStr(0, 34, l2);
  } while (u8g2.nextPage());
}

void drawResult(float glucose, float voltage_mV) {
  char gtxt[10];
  char vtxt[12];

  dtostrf(glucose, 5, 1, gtxt);
```

```
  dtostrf(voltage_mV, 6, 1, vtxt);

  u8g2.firstPage();
  do {
    u8g2.setFont(u8g2_font_6x10_tr);
    u8g2.drawStr(0, 10, "GLUCOSE");

    u8g2.setFont(u8g2_font_10x20_tr);
    u8g2.drawStr(0, 34, gtxt);

    u8g2.setFont(u8g2_font_6x10_tr);
    u8g2.drawStr(70, 34, "mg/dL");

    u8g2.drawStr(0, 50, "mV:");
    u8g2.drawStr(38, 50, vtxt);

    u8g2.drawStr(0, 63, classify(glucose));
  } while (u8g2.nextPage());
}
```

Listing 1: Code for Glucometer - Initialization of variables

This first code snippet (listing 1) initializes variables and designs the interface for the OLED display. An important part of the code is the linear calibration code. We set the slope and offset, as the relationship between voltage and glucose is fairly linear, following the expression below:

$$glucose = voltage \cdot SLOPE + OFFSET$$

This second code snippet (listing 2) uses

```
    analogReference(INTERNAL)
```

because the Arduino Nano has a reference voltage of 1.1V.

```
void setup() {
  Serial.begin(9600);

  analogReference(INTERNAL); //the analog reference
      is about 1.1V. We will use this because our
      sensor is in millivolt range. We know this
      because
//we used a multimeter and we obtained a voltage of
    2.5V.
  u8g2.begin();
  //These lines do linear calibration
  SLOPE = (G2 - G1) / (V2 - V1);
  OFFSET = G1 - (SLOPE * V1);

  //Starting the serial monitor
  Serial.println("Serial monitor ready");

  drawText("Glucometer ready","");
}
```

Listing 2: Code for Glucometer - Void setup

In the next code snippet in listing 3, we debugged to see if the raw values were maxed out, as we were having trouble reading the reactive strip and measuring the current passing through the strip working electrode. We also printed the results on the serial monitor.

```
//Reading voltage and calibrating those values to
    convert them to readable glucose levels
void loop() {

  drawText("Measuring...", "");
  delay(500);

//These lines help us debug the code and see if the
    raw values are maxed out and if there is a
    connection issue.
//If Raw ADC: 1023, the input is saturated. If we
    see random values jumping around, the input is
    floating.
//We want the raw ADC to have stable values.
  int raw_debug = analogRead(SENSOR);
  Serial.print("Raw ADC: ");
  Serial.println(raw_debug);

  float voltage_mV = readVoltage_mV(32);
  float glucose = voltageToGlucose(voltage_mV);

  // glucose = constrain(glucose, 0, 500); (
      commented out line to debug)

//Printing voltage and glucose levels on serial
    monitor
  Serial.print("Voltage: ");
  Serial.print(voltage_mV, 1);
  Serial.print(" mV | Glucose: ");
  Serial.print(glucose, 1);
  Serial.println(" mg/dL");

  drawResult(glucose, voltage_mV);

  delay(3000);

  drawText("Glucometer ready", "");
}
```

Listing 3: Code for Glucometer - Void loop

Conclusion and Reflection

To conclude our glucometer project, we conducted a large number of tests to verify its reliability. For comparison, we used a real glucose meter as a reference. Although at first the device only read a value of 500 mg/dL, which is extremely high and unrealistic, by adjusting the boundaries and refining our calibration, we were able to obtain results much closer to the actual model.

The system was showing 500 mg/dL because it maxed out, probably due to the use of the op amp LM741. This model is hitting its limit, as the OP-amp is not operating in the most suitable feedback loop. As a result, the signal exceeds the measurable range of the Arduino's ADC, causing the readings to clip at their maximum value. This highlights the importance of selecting an appropriate operational amplifier and properly designing the transimpedance configuration to ensure reliable and precise measurements. Overall, this project provided valuable hands-on experience in microcontroller systems, analog signal conditioning, and sensor interfacing, while also highlighting the gap between a functional prototype and a medically reliable device.

Appendix

Some code was produced with the help of intelligent chatbots such as ChatGPT, so its work and credit is accounted for via written evidence such as this text. Specific code snippets will also be commented on if additional AI help was required. We worked on improving and understanding the first version of

the code proposed by the AI, so we could adjust it to our own requirements.

A. Pictures and Screenshots

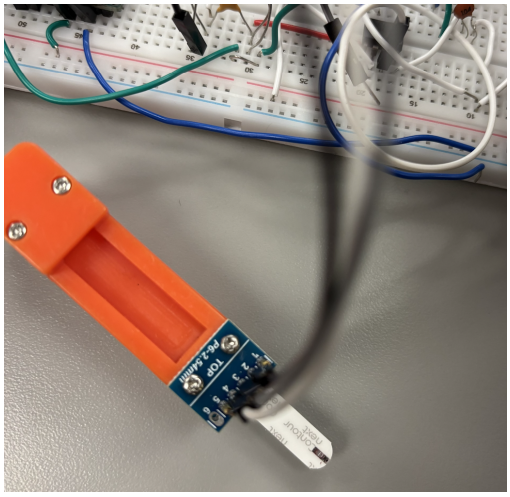

Fig. 3: PCB Clip with the correspondent Reactive Probe

Fig. 4: OLED display connected to the Arduino-based glucometer system